

gmapping Mapping Algorithm

gmapping Mapping Algorithm

1. Course Content
2. gmapping Algorithm Introduction
3. Running the Demo
 - 3.1 Mapping Process
 - 3.2 Saving the Map
 - 3.3 Configuration File

1. Course Content

Basics: run the demo program and use the gmapping SLAM algorithm to build a map and save an occupancy grid map.

Advanced: understand the configuration parameters of the gmapping algorithm.

2. gmapping Algorithm Introduction

- gmapping only works when the number of 2D laser points in a single frame is less than 1440. If a single frame contains more than 1440 laser points, a problem like `[mapping-4] process has died` will occur.
- Gmapping is a commonly used open-source SLAM algorithm based on the filtering SLAM framework.
- Gmapping is based on the RBPF particle filter algorithm, which separates the simultaneous localization and mapping process: localization is performed first, then mapping.
- Gmapping makes two main improvements to the RBPF algorithm: an improved proposal distribution and selective resampling.

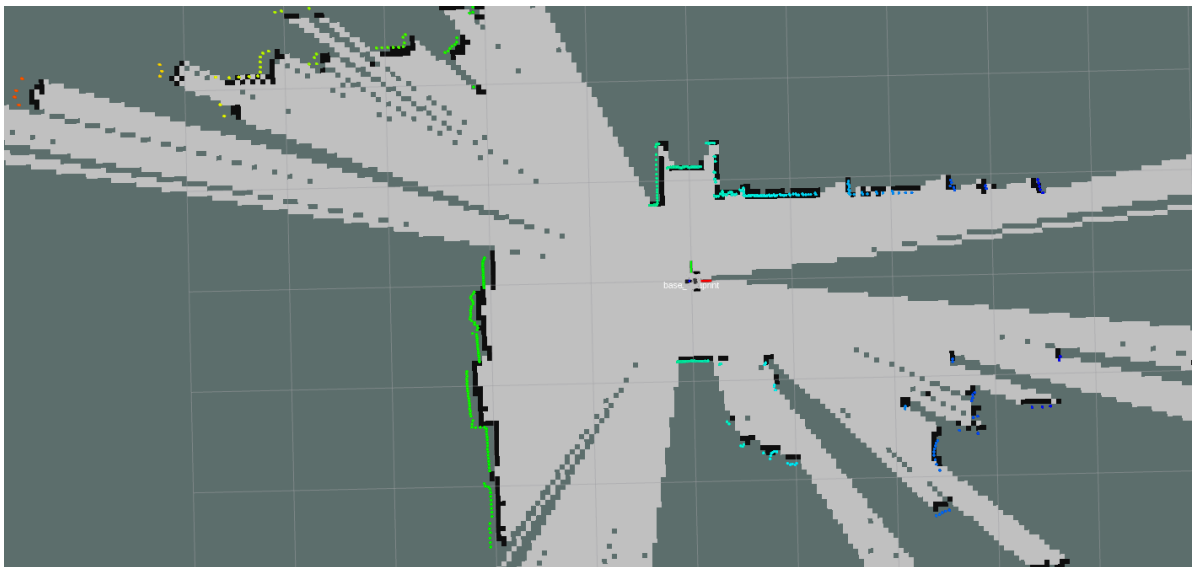
Advantages: Gmapping can build indoor maps in real time, and building maps of small scenes requires relatively little computation while achieving high accuracy.

Disadvantages: As the scene grows larger, the number of required particles increases. Because each particle carries a map, both the memory and

computational cost increase when building large maps. Therefore, it is not suitable for mapping large scenes. Moreover, it has no loop-closure detection, so the map may become misaligned when a loop closes; although increasing the number of particles can close the loop, it comes at the cost of higher computation and memory usage.

3. Running the Demo

```
ros2 launch microros_v2_bringup gmapping.launch.py
```



3.1 Mapping Process

- Note: when mapping, slower speeds give better results (especially the rotation speed should be slow). If you move too fast, the mapping quality will be poor.
- Control the robot to move and complete mapping of the environment. You can use keyboard control or gamepad control; keyboard control is used here as an example.

```
ros2 run common_demo keyboard_control
```

```

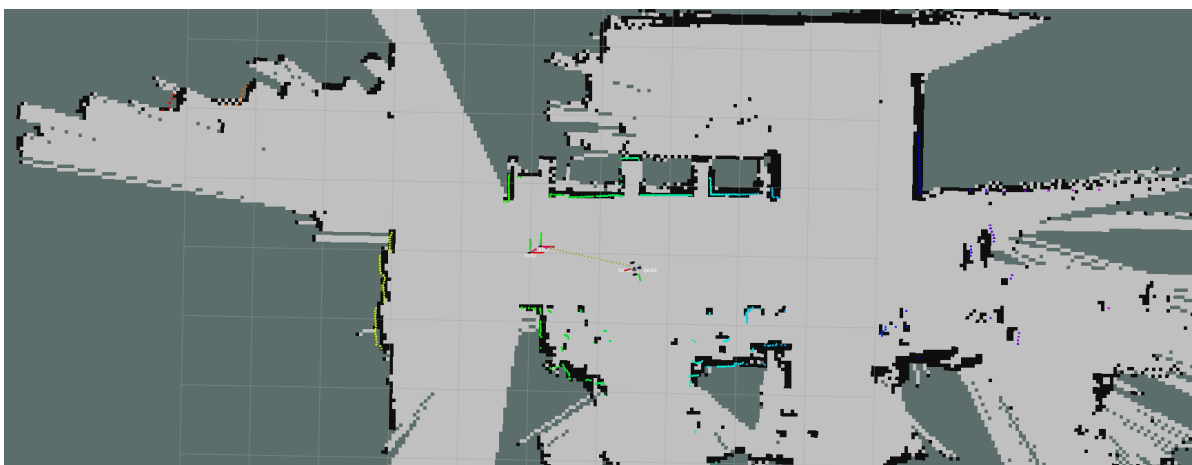
yahboom@yahboom:~$ ros2 run common_demo keyboard_control

Reading from the keyboard and Publishing to Twist!
-----

Moving around:
   u   i   o
   j   k   l
   m   ,   .

For Holonomic mode (strafing), hold down the shift key

```



3.2 Saving the Map

- Run the following command in the terminal to save the occupancy grid map:

```
ros2 launch microros_v2_bringup save_map.launch.py
```

[!TIP]

Optional parameters:

`map_name`: specify the name of the map to save; the default value is map

For example, to save a map named test.yaml, add the parameter: `ros2 launch`

```
microros_v2_bringup save_map.launch.py map_name:=test
```

A **Map saved successfully** message in the terminal means the map was saved successfully. If saving fails, try again.

The default map save path is as follows:

```
~/map/map.yaml
```

One PGM image and one YAML file, yahboom_map.yaml

```
image: yahboom_map.pgm
mode: trinary
resolution: 0.05
origin: [-10, -10, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

Parameter explanations:

- `image`: path to the map file; it can be an absolute path or a relative path
- `mode`: this attribute can be one of trinary, scale, or raw, depending on the mode chosen; trinary is the default mode
- `resolution`: the resolution of the map, in meters/pixel
- `origin`: the 2D pose (x, y, yaw) of the bottom-left corner of the map. Here yaw is rotated counterclockwise (yaw=0 means no rotation). Many parts of the current system ignore the yaw value.
- `negate`: whether to invert the meaning of white/black and free/occupied (the interpretation of the thresholds is not affected)
- `occupied_thresh`: pixels with an occupancy probability greater than this threshold are considered fully occupied.
- `free_thresh`: pixels with an occupancy probability less than this threshold are considered completely free.

3.3 Configuration File

- Configuration file path:

```
~/microros_ws/src/microros_v2_bringup/config/slam_gmapping.yaml
```

- Meanings of the default configuration parameters:

```
slam_gmapping:
  ros__parameters:
    # Angular update threshold (radians): the map is updated when the robot
    # rotates beyond this angle
    angularUpdate: 0.25
    # Angular sampling step (radians): angular search step for scan matching
    astep: 0.05
    # Robot base frame name
    base_frame: base_footprint
    # Map frame name
    map_frame: map
    # Odometry frame name
    odom_frame: odom
    # Scan matching iteration termination condition: stop iterating when the
    # parameter change is below this value
    delta: 0.05
    # Maximum number of scan matching iterations
    iterations: 5
    # Kernel size for scan matching, affects matching smoothness
    kernelSize: 1

    # Distance sampling range for large-scale scan matching (m)
    lasamplerange: 0.005
    # Distance sampling step for large-scale scan matching (m)
    lasamplestep: 0.005
    # Linear update threshold (m): the map is updated when the robot moves beyond
    # this distance
    linearUpdate: 0.5
    # Distance sampling range for small-scale scan matching (m)
    llsamplerange: 0.01
    # Distance sampling step for small-scale scan matching (m)
    llsamplestep: 0.01

    # Measurement noise covariance of the laser scan
    lsigma: 0.075
    # Number of laser beams to skip: 0 means using all laser points
    lskip: 0
    # Linear sampling step (m): linear search step for scan matching
    lstep: 0.05
    # Map update time interval (s)
    map_update_interval: 3.0
    # Maximum laser range (m): points beyond this distance are ignored
    maxRange: 6.0
    # Effective maximum laser range (m): maximum distance used for mapping
    maxUrange: 4.0
    # Minimum score threshold for scan matching: matches below this value are
    # discarded
    minimum_score: 0.0
```

```

# Occupancy probability threshold: cells above this value are considered
occupied
occ_thresh: 0.25
# Gain factor: affects the map update speed
ogain: 3.0
# Number of particles in the particle filter
particles: 30
# QoS (Quality of Service) parameter settings
qos_overrides:
  /parameter_events:
    publisher:
      depth: 1000          # Message queue depth
      durability: volatile # Durability policy
      history: keep_all    # History policy
      reliability: reliable # Reliability policy
  /tf:
    publisher:
      depth: 1000
      durability: volatile
      history: keep_last  # Keep only the latest message
      reliability: reliable
# Resampling threshold: resampling is triggered when the effective particle
count falls below this ratio
resampleThreshold: 0.5
# Angular noise covariance of scan matching
sigma: 0.05
# Odometry model parameter: linear error ratio caused by linear motion
srr: 0.1
# Odometry model parameter: angular error ratio caused by linear motion
srt: 0.2
# Odometry model parameter: linear error ratio caused by angular motion
str: 0.1
# Odometry model parameter: angular error ratio caused by angular motion
stt: 0.2
# Time update interval (s): the map is updated even if the robot does not
move
temporalUpdate: 1.0
# Coordinate transform publish period (s)
transform_publish_period: 0.05
# Whether to use simulated time
use_sim_time: false
# Maximum range of the map along the x axis (m)
xmax: 100.0
# Minimum range of the map along the x axis (m)
xmin: -100.0
# Maximum range of the map along the y axis (m)
ymax: 100.0
# Minimum range of the map along the y axis (m)
ymin: -100.0

```